Edintel S.A.

Sistema: VigiNurse

Manual técnico

Versión del sistema: 1.0

Versión del manual: 1.0

Fecha de creación: 12/05/2026

Autores:

Brandon Rodriguez Vega

Mariano Oconitrillo Vega

Índice

Versiones de componentes.....5
Frontend web..... 5
Frontend móvil.....6
Backend..... 7
Instalación del Frontend Web en Windows..... 8
Paso 1 — Pre Requisitos en Windows.....8
1.1 Node.js 22 (LTS).....8
1.2 Git (si no lo tienes)..... 8
Paso 2 — Clonar el repositorio..... 8
Paso 3 — Instalar dependencias..... 8
Paso 4 — Configurar variables de entorno..... 9
Paso 5 — Verificar que el backend esté corriendo..... 9
Paso 6 — Ejecutar el proyecto en desarrollo.....9
Instalación del Frontend Móvil en Windows.....10
Paso 1 — Pre-requisitos en Windows.....10
1.1 Node.js 22 (LTS)..... 10
1.2 Git (si no lo tienes)..... 10
1.3 Expo Go (recomendado para probar en un celular físico)..... 10
Paso 2 — Clonar el repositorio..... 10
Paso 3 — Instalar dependencias..... 10
Paso 4 — Configurar variables de entorno..... 11
4.1 Obtener la IP local de tu PC.....11
4.2 Editar el .env.....11
Paso 5 — Verificar que el backend acepte conexiones desde la red.....11
Paso 6 — Ejecutar el proyecto.....12
Paso 7 — Abrir la app..... 12
Instalación del backend en Windows.....13
Paso 1 — Pre Requisitos en Windows.....13
1.1 .NET 8 SDK.....13
1.2 PostgreSQL 14+.....13
1.3 Mosquitto (MQTT)..... 13
Paso 2 — Clonar el repositorio..... 13
Paso 3 — Configurar PostgreSQL..... 13
Paso 4 — Configurar appsettings.json.....14
Paso 5 — Configurar Mosquitto..... 14
5.1 Editar la configuración.....14
5.2 Iniciar el servicio.....14
5.3 Verificar..... 14
Paso 6 — Orthanc nativo..... 15
6.1 Instalación..... 15
6.2 Verificar..... 15
6.3 Archivo de configuración.....15
6.4 Registrar Orthanc en la BD..... 15

Paso 7 — EF Core CLI y migraciones..... 16
Paso 8 — Ejecutar el API..... 16
Orthanc con Docker en Windows..... 17
Paso 1 — Instalar Docker Desktop en Windows..... 17
1.1 Instalación..... 17
1.2 Verificar Docker..... 17
Paso 2 — Configurar y levantar Orthanc con Docker..... 17
2.1 Crear estructura de carpetas..... 17
2.2 Crear el archivo de configuración orthanc.json..... 18
2.3 Levantar el contenedor..... 18
2.4 Verificar que Orthanc está corriendo..... 19
2.5 Comandos de gestión del contenedor..... 19
Paso 3 — Instalar Python y generar imagen DICOM de prueba..... 19
3.1 Instalar Python..... 19
3.2 Instalar librerías..... 20
3.3 Script generar_dicom.py..... 20
3.4 Ejecutar el script..... 22
3.5 Verificar en Orthanc..... 22
Paso 4 — Flujo completo de pruebas con Swagger..... 22
4.1 Login y autenticación..... 22
4.2 Registrar la configuración PACS..... 22
4.3 Vincular paciente con Orthanc..... 23
4.4 Ver estudios del paciente (tiempo real)..... 23
4.5 Registrar estudio en la BD..... 23
4.6 Ver detalle del estudio..... 23
Configuración del Visor OHIF para Imágenes Médicas..... 24
Paso 1 — Configurar nginx como proxy inverso..... 24
1.1 Crear estructura de carpetas..... 25
1.2 Crear el archivo default.conf..... 25
1.1 Levantar nginx..... 26
1.1 Verificar..... 26
Paso 2 — Levantar el visor OHIF..... 26
2.1 Levantar el contenedor..... 26
2.2 Verificar..... 26
Paso 3 — Configurar el Frontend..... 27
2.2 Variable de entorno..... 27
MQTT: Conectar Gateway con Tag..... 28
Requisitos previos..... 28
Paso 1 — Configurar el Gateway MKGW3..... 28
1.1 Encender el gateway..... 28
1.2 Verificar los LEDs..... 28
1.3 Configurar la app MKScannerPro..... 29
1.4 Conectar al gateway desde la app..... 29
1.5 Configurar Network Settings..... 30

1.6 Configurar MQTT Settings..... 30
1.7 Conectar y verificar LEDs.....31
Paso 2 — Verificar que los datos del Tag L02 llegan.....31
Paso 3 — Conectar el API .NET de VigiNurse.....32
3.1 Librería MQTTnet..... 32
3.2 Configuración en appsettings.json..... 32
Paso 4 — Procedimiento al cambiar de red.....32
Resumen de puertos.....33

Versiones de componentes

Frontend web

Componente	Versión
Node.js	22.x (LTS)
Next.js	16.1.6
React	19.2.3
React DOM	19.2.3
TypeScript	5
Tailwind CSS	4
daisyUI	5.5.19
@reduxjs/toolkit	2.11.2
react-redux	9.2.0
axios	1.13.6
@microsoft/signalr	10.0.0
toastify-js	1.12.0
cally	0.9.2
ESLint	9
Playwright (testing)	1.59.1
OHIF Viewer (Docker)	ohif/app:latest
nginx (Docker)	nginx:alpine

Al ejecutar npm install por primera vez, npm descarga automáticamente todos los paquetes listados en package.json, desde el registro de npm. No se necesita instalar nada de npm manualmente.

Lo que sí debes instalar/configurar manualmente son las herramientas y servicios externos:

Node.js 22 (el sistema puede traer una versión más vieja por defecto, usar nvm install 22 && nvm use 22)
Docker Desktop (para correr los contenedores de OHIF y nginx)
Variable de entorno .env.local con NEXT_PUBLIC_OHIF_URL=http://localhost:8080

Nota sobre Node.js: a diferencia de los paquetes NuGet del backend, Node.js no se descarga solo, es el runtime que ejecuta npm y next. Si no está instalado (o está en una versión muy vieja), npm install y npm run dev van a fallar directamente.

Frontend móvil

Componente	Versión
Node.js	22.x (LTS)
Expo SDK	54.0.33
Expo Router	6.0.23
React	19.1.0
React Native	0.81.5
React Navigation (drawer)	7.5.0
react-native-paper	5.12.5
@reduxjs/toolkit	2.3.0
react-redux	9.1.2
axios	1.7.7
@microsoft/signalr	10.0.0
@react-native-async-storage/async-storage	2.2.0
expo-secure-store	15.0.8
expo-notifications	0.32.17
expo-camera	17.0.10
TypeScript	5.3.3
ESLint	8.57.0

Backend

Componente	Versión
.NET	8.0
ASP.NET Core	8.0
Entity Framework Core	compatible con 8.0
PostgreSQL	14 o superior
Orthanc	1.12.x (última estable)
Mosquitto (MQTT broker)	2.x
MailKit (NuGet)	4.7.1.1
JwtBearer (NuGet)	8.0.0
Identity.EntityFrameworkCore (NuGet)	8.0.0
MQTTnet (NuGet)	4.3.6.1152
QRCoder (NuGet)	1.7.0

Al ejecutar dotnet run o dotnet build por primera vez, .NET descarga automáticamente todos los paquetes NuGet listados en el archivo .csproj desde nuget.org. No se necesita instalar nada de NuGet manualmente.
Los paquetes NuGet se descargan solos: MailKit, MQTTnet, Entity Framework, JWT, QRCoder, Swashbuckle y todos los demás paquetes de la tabla de versiones se descargan automáticamente con el primer dotnet run.

Lo que sí debes instalar manualmente son los servicios externos, como: PostgreSQL, Mosquitto, Orthanc y .NET 8 SDK.

Instalación del Frontend Web en Windows

Paso 1 — Pre Requisitos en Windows

Instala los siguientes componentes en el orden indicado.

1.1 Node.js 22 (LTS)

Ingresa a https://nodejs.org/en/download y descarga el instalador .msi para Windows x64 (versión 22.x LTS).

Ejecuta el instalador con opciones por defecto (incluye npm).
Verifica la instalación:

```
node -v
npm -v
```

Debe mostrar v22.x.x para Node y una versión 10.x o superior para npm. Si tu equipo ya tiene una versión vieja de Node instalada (por ejemplo v12 o v16), desinstálala primero o usa nvm-windows para manejar varias versiones.

1.2 Git (si no lo tienes)

Ingresa a https://git-scm.com/download/win y ejecuta el instalador con opciones por defecto.

Paso 2 — Clonar el repositorio

```
git clone <URL_DEL_REPOSITORIO> VigiNurse-FrontEnd
cd VigiNurse-FrontEnd
```

Paso 3 — Instalar dependencias

```
npm install
```

Esto descarga automáticamente todos los paquetes listados en package.json: Next.js, React, Tailwind CSS, daisyUI, Redux Toolkit, axios, SignalR, etc. No se necesita instalar nada manualmente aparte de Node.js. La primera vez puede tardar varios minutos.

Paso 4 — Configurar variables de entorno

Crea un archivo .env.local en la raíz del proyecto (VigiNurse-FrontEnd\.env.local):

NEXT_PUBLIC_API_BASE_URL=http://localhost:5025/api

Si además vas a usar el visor de imágenes médicas (OHIF), agrega también:

NEXT_PUBLIC_OHIF_URL=http://localhost:8080

Paso 5 — Verificar que el backend esté corriendo

El frontend necesita que el backend (VigiNurse.API) esté levantado en http://localhost:5025 (Paso 8 del setup del backend: dotnet run).

Verifica que el backend permita peticiones desde el frontend revisando appsettings.json:

```
"App": {
  "UrlFrontend": "http://localhost:3000"
}
```

Esto debe coincidir con la URL donde correrá el frontend (Paso 6) para que CORS funcione correctamente.

Paso 6 — Ejecutar el proyecto en desarrollo

npm run dev
El frontend levanta en:

http://localhost:3000
Ábrelo en el navegador, debería mostrar la página de login de VigiNurse.

Instalación del Frontend Móvil en Windows

Instala los siguientes componentes en el orden indicado.

Paso 1 — Pre-requisitos en Windows

1.1 Node.js 22 (LTS)

Ingresa a https://nodejs.org/en/download y descarga el instalador .msi para Windows x64 (versión 22.x LTS).

Verifica la instalación:

```
node -v
npm -v
```

1.2 Git (si no lo tienes)

Ingresa a https://git-scm.com/download/win y ejecuta el instalador con opciones por defecto.

1.3 Expo Go (recomendado para probar en un celular físico)

Instala la app Expo Go desde Play Store (Android) o App Store (iOS) en tu teléfono. Es la forma más rápida de probar la app sin compilar nada nativo.

Paso 2 — Clonar el repositorio

```
git clone <URL_DEL_REPOSITORIO> VigiNurse-FrontEnd-Mobile
cd VigiNurse-FrontEnd-Mobile
```

Paso 3 — Instalar dependencias

```
npm install
```

Esto descarga automáticamente todos los paquetes listados en package.json: Expo SDK, React Native, Redux Toolkit, axios, SignalR, etc.

Paso 4 — Configurar variables de entorno

Copia el archivo de ejemplo:

copy .env.example .env

Esto crea un .env con:

- EXPO_PUBLIC_API_BASE_URL=http://localhost:5025/api
- EXPO_PUBLIC_WS_URL=ws://localhost:5025

⚠ Importante - localhost NO funciona en mobile: a diferencia del frontend web (que corre en el mismo navegador/PC que el backend), la app mobile corre en otro dispositivo (tu celular o un emulador). localhost desde el celular se refiere al propio celular, no a tu PC. Tienes que usar la IP local de tu PC en la red Wi-Fi.

4.1 Obtener la IP local de tu PC

ipconfig

Busca la sección de tu adaptador Wi-Fi (Wireless LAN adapter Wi-Fi) y anota la Dirección IPv4 (ej: 192.168.100.7).

4.2 Editar el .env

- EXPO_PUBLIC_API_BASE_URL=http://192.168.100.7:5025/api
- EXPO_PUBLIC_WS_URL=ws://192.168.100.7:5025

Reemplaza 192.168.100.7 por la IP que obtuviste en el paso anterior. Tu celular y tu PC deben estar conectados a la misma red Wi-Fi.

Paso 5 — Verificar que el backend acepte conexiones desde la red

El backend debe estar corriendo y escuchando en todas las interfaces (no solo localhost), para que el celular pueda alcanzarlo:

dotnet run --urls "http://0.0.0.0:5025"

Si Windows Firewall pregunta si permitir el acceso a la red, selecciona Permitir (al menos para redes privadas).

Verifica desde el navegador del celular que http://192.168.100.7:5025/swagger cargue Swagger, si esto no carga, el resto tampoco va a funcionar (revisa firewall/antivirus).

Paso 6 — Ejecutar el proyecto

npx expo start

Esto abre Expo Dev Tools en la terminal con un código QR.

Paso 7 — Abrir la app

- Celular físico: abre la app Expo Go y escanea el código QR que aparece en la terminal.
- Emulador Android: con Android Studio abierto y un AVD corriendo, presiona a en la terminal de Expo.
- Web (opcional, para pruebas rápidas): presiona w en la terminal , abre http://localhost:19006 en el navegador (en este caso localhost sí funciona porque corre en el mismo PC).

Instalación del backend en Windows

Paso 1 — Pre Requisitos en Windows

Instala los siguientes componentes en el orden indicado.

1.1 .NET 8 SDK

- Ingresa a https://dotnet.microsoft.com/download/dotnet/8.0
- Descarga el instalador .exe para tu arquitectura (x64 recomendado) y ejecútalo.
- Verifica la instalación: –dotnet version

1.2 PostgreSQL 14+

- Ingresa a https://www.postgresql.org/download/windows/ y descarga el instalador.
- Durante la instalación, anota bien la contraseña que configures para el usuario postgres — la necesitarás más adelante.
- Deja el puerto por defecto (5432). Se recomienda instalar pgAdmin (incluido en el instalador).

1.3 Mosquitto (MQTT)

- Ingresa a https://mosquitto.org/download/ y descarga el instalador .exe para Windows.
- Ejecuta el instalador con opciones por defecto. Mosquitto se instala como servicio de Windows.

Paso 2 — Clonar el repositorio

git clone <URL_DEL_REPOSITORIO> VigiNurse-BackEnd
cd VigiNurse-BackEnd\VigiNurse.API

Paso 3 — Configurar PostgreSQL

- Abre pgAdmin, conéctate con el usuario postgres.
- Clic derecho en 'Login/Group Roles' → 'Create' → crea el usuario viginurse_user con contraseña 12345678.
- Clic derecho en 'Databases' → 'Create' → crea la base de datos viginurse con propietario viginurse_user.

Paso 4 — Configurar appsettings.json

```
"ConnectionStrings": {
  "DefaultConnection":
"Host=localhost;Database=viginurse;Username=viginurse_user;Password=12345678"
},
"Mqtt": {
  "Broker": "localhost",
  "Port": "1883"
},
"Orthanc": {
  "UsuarioDefault": "orthanc",
  "PasswordDefault": "orthanc"
}
```

Paso 5 — Configurar Mosquitto

5.1 Editar la configuración

Navega a `C:\Program Files\mosquitto\` y abre `mosquitto.conf` con el Bloc de notas (clic derecho → Abrir con).
Agrega estas dos líneas al final del archivo y guarda:

```
listener 1883
allow_anonymous true
```

5.2 Iniciar el servicio

Abre Servicios (Win+R → services.msc), busca Mosquitto Broker, pon el Tipo de inicio en Automático y haz clic en Iniciar.
O desde CMD como administrador:

```
net stop mosquitto
net start mosquitto
```

5.3 Verificar

Abre dos ventanas de CMD. En la primera:

```
cd "C:\Program Files\mosquitto"
mosquitto_sub.exe -h localhost -t /test
```

En la segunda:

```
cd "C:\Program Files\mosquitto"
mosquitto_pub.exe -h localhost -t /test -m "hola"
```

Debes ver el mensaje 'hola' aparecer en la primera ventana.

Paso 6 — Orthanc nativo

Esta sección describe la instalación nativa de Orthanc en Windows. Si prefieres usar Docker (más simple y recomendado para desarrollo), ve a la Sección 3 de este documento.

6.1 Instalación

- Ingresa a https://www.orthanc-server.com/download/windows/ y descarga el instalador estable (1.12.x).
- Ejecuta el instalador con opciones por defecto. Orthanc se instala como servicio de Windows en el puerto 8042.

6.2 Verificar

Abre el navegador en http://localhost:8042 — usuario: orthanc / contraseña: orthanc. O desde CMD:

```
curl http://localhost:8042/system -u orthanc:orthanc
```

6.3 Archivo de configuración

El archivo de configuración en Windows se encuentra en:

```
C:\Program Files\Orthanc Server\Configuration\orthanc.json
```

Valores que deben estar presentes (ya vienen por defecto):

```
{
  "HttpPort": 8042,
  "RemoteAccessAllowed": true,
  "AuthenticationEnabled": true,
  "RegisteredUsers": { "orthanc": "orthanc" }
}
```

6.4 Registrar Orthanc en la BD

Una vez que el API esté corriendo (Paso 8), ejecuta desde Swagger:

```
POST /api/configuracion-pacs
{
  "hospitalId": 1,
  "nombrePACS": "PACS Principal",
  "orthancUrl": "http://localhost:8042",
  "activo": true
}
```

Paso 7 — EF Core CLI y migraciones

Instala la herramienta global de Entity Framework (solo una vez por máquina):

```
dotnet tool install --global dotnet-ef
```

Si el comando no se reconoce después de instalarlo, cierra y vuelve a abrir el terminal. Windows necesita refrescar las variables de entorno.

Aplica las migraciones:

```
cd C:\ruta\al\proyecto\VigiNurse.API
dotnet ef database update
```

Paso 8 — Ejecutar el API

Ejecuta:

```
dotnet run
```

El API levanta en http://localhost:5000. Para especificar puerto o exponer en red:

```
dotnet run --urls "http://0.0.0.0:5000"
```

Swagger disponible en: http://localhost:5000/swagger

Orthanc con Docker en Windows

Con Docker, el servidor PACS corre en un contenedor aislado con un solo comando, sin instalar dependencias ni configurar servicios de Windows manualmente. Es la forma recomendada para entornos de desarrollo.

Paso 1 — Instalar Docker Desktop en Windows

1.1 Instalación

- Ingresa a https://www.docker.com/products/docker-desktop/ y descarga Docker Desktop para Windows (versión estable).
- Ejecuta el instalador. Cuando te pregunte, selecciona WSL 2 (recomendado).
- Al finalizar, reinicia el equipo si el instalador lo solicita.
- Abre Docker Desktop y espera a que el ícono de la ballena en la barra de tareas quede en color sólido, eso indica que Docker está listo.

Docker Desktop en Windows requiere WSL 2. Si tu sistema no lo tiene, Docker Desktop te guiará para instalarlo automáticamente durante la primera ejecución.

1.2 Verificar Docker

Ejecuta:

```
docker --version

# Prueba completa — debes ver 'Hello from Docker!'
docker run hello-world
```

Paso 2 — Configurar y levantar Orthanc con Docker

2.1 Crear estructura de carpetas

```
mkdir %USERPROFILE%\orthanc-test
mkdir %USERPROFILE%\orthanc-test\db
```

Esto crea la carpeta orthanc-test en tu directorio de usuario (ej: C:\Users\TuUsuario\orthanc-test) y una subcarpeta db donde Orthanc almacenará las imágenes DICOM de forma persistente.

2.2 Crear el archivo de configuración orthanc.json

Ejecuta este bloque completo en PowerShell de una sola vez:

powershell

```
@"
{
  "Name": "VigiNurse Test PACS",
  "HttpServerEnabled": true,
  "HttpPort": 8042,
  "DicomServerEnabled": true,
  "DicomPort": 4242,
  "RemoteAccessAllowed": true,
  "AuthenticationEnabled": true,
  "RegisteredUsers": { "orthanc": "orthanc" },
  "StorageDirectory": "/var/lib/orthanc/db",
  "DicomWeb": {
    "Enable": true,
    "Root": "/dicom-web/",
    "EnableWado": true,
    "WadoRoot": "/wado"
  }
}
"@ | Out-File -FilePath "$env:USERPROFILE\orthanc-test\orthanc.json" -Encoding
UTF8
```

Alternativa: crea el archivo manualmente con el Bloc de notas, pega el JSON dentro y guárdalo como orthanc.json en la carpeta orthanc-test (codificación UTF-8).

2.3 Levantar el contenedor

Ejecuta este comando completo en CMD (el símbolo ^ divide el comando en varias líneas):

```
docker run -d ^
  --name orthanc-test ^
  -p 8042:8042 ^
  -p 4242:4242 ^
  -v %USERPROFILE%\orthanc-test\orthanc.json:/etc/orthanc/orthanc.json:ro ^
  -v %USERPROFILE%\orthanc-test\db:/var/lib/orthanc/db ^
  orthancteam/orthanc
```

En PowerShell usa ` (backtick) en lugar de ^ para dividir el comando.

La primera vez Docker descargará la imagen de Orthanc (~200 MB). Puede tomar varios minutos según tu conexión. Las siguientes ejecuciones serán instantáneas.

2.4 Verificar que Orthanc está corriendo

```
# El contenedor debe aparecer con estado 'Up'
docker ps

# Verificar respuesta HTTP
curl -u orthanc:orthanc http://localhost:8042/system
```

También puedes abrir el navegador en http://localhost:8042 — usuario: orthanc / contraseña: orthanc.

2.5 Comandos de gestión del contenedor

```
# Detener Orthanc
docker stop orthanc-test

# Iniciar Orthanc (después de detenerlo)
docker start orthanc-test

# Ver logs en tiempo real (útil para depurar)
docker logs -f orthanc-test

# Eliminar el contenedor (los datos en la carpeta db se conservan)
docker rm -f orthanc-test
```

Persistencia de datos: Las imágenes DICOM se guardan en `%USERPROFILE%\orthanc-test\db` en tu disco local, no dentro del contenedor. Aunque elimines y recrees el contenedor, los datos persisten.

Paso 3 — Instalar Python y generar imagen DICOM de prueba

Un archivo DICOM tiene los datos del paciente embebidos. Si descargas un DICOM de internet, su cédula no coincidirá con la de tu BD. Por eso debes generar uno con la cédula exacta del paciente en VigiNurse, así el API lo encuentra correctamente al buscar en Orthanc.

3.1 Instalar Python

- Ingresa a https://www.python.org/downloads/ y descarga Python 3.11 o superior.
- IMPORTANTE: en la primera pantalla del instalador, marca la casilla 'Add Python to PATH' antes de hacer clic en Install Now.

```
python --version
 pip --version
```

3.2 Instalar librerías

Ejecuta:

```
pip install pydicom requests
```

3.3 Script generar_dicom.py

Crea un archivo llamado `generar_dicom.py` en tu carpeta de trabajo y pega el siguiente contenido:

```
import pydicom
import pydicom.uid
import requests
from datetime import datetime

# ——— Datos del paciente en VigiNurse
————————————————————————————————————
CEDULA    = "1234567890"      # debe coincidir con Paciente.Cedula en la BD
NOMBRE    = "PEREZ^JUAN"      # formato DICOM: APELLIDO^NOMBRE
(mayúsculas)

# ——— Orthanc
————————————————————————————————————————————————————————————————————
————————————
ORTHANC_URL  = "http://localhost:8042"
ORTHANC_USER = ("orthanc", "orthanc")

# ——— Crear dataset DICOM mínimo
————————————————————————————————————————————————
ds = pydicom.Dataset()
ds.file_meta = pydicom.Dataset()
ds.file_meta.MediaStorageSOPClassUID    =
pydicom.uid.SecondaryCaptureImageStorage
ds.file_meta.MediaStorageSOPInstanceUID = pydicom.uid.generate_uid()
ds.file_meta.TransferSyntaxUID          = pydicom.uid.ExplicitVRLittleEndian
ds.is_implicit_VR   = False
ds.is_little_endian = True

# Tags del paciente
ds.PatientID        = CEDULA          # (0010,0020) → cédula
ds.PatientName      = NOMBRE          # (0010,0010) → nombre
ds.PatientBirthDate = "19900101"
ds.PatientSex       = "M"

# Tags del estudio
```

```
ds.StudyInstanceUID  = pydicom.uid.generate_uid()
ds.SeriesInstanceUID = pydicom.uid.generate_uid()
ds.SOPInstanceUID    = pydicom.uid.generate_uid()
ds.SOPClassUID       = pydicom.uid.SecondaryCaptureImageStorage
ds.StudyDate         = datetime.now().strftime("%Y%m%d")
ds.Modality          = "CR"      # CR = Computed Radiography
ds.StudyDescription  = "Radiografia de torax PA"

# Imagen mínima válida (1x1 pixel negro)
ds.Rows    = 1
ds.Columns = 1
ds.SamplesPerPixel          = 1
ds.PhotometricInterpretation = "MONOCHROME2"
ds.BitsAllocated = 8
ds.BitsStored    = 8
ds.HighBit       = 7
ds.PixelRepresentation = 0
ds.PixelData = bytes([0])

# Guardar y subir
pydicom.dcmwrite("prueba.dcm", ds, write_like_original=False)
print("✓ prueba.dcm generado")

with open("prueba.dcm", "rb") as f:
    r = requests.post(
        f"{ORTHANC_URL}/instances",
        data=f.read(),
        headers={"Content-Type": "application/dicom"},
        auth=ORTHANC_USER
    )

print(f"✓ Subido a Orthanc: {r.status_code}")
print(r.json())
```

Edita CEDULA con la cédula exacta del paciente en tu BD (campo Paciente.Cedula). Edita NOMBRE con el formato DICOM: APELLIDO^NOMBRE en mayúsculas. Ejemplo: MORA^CARLOS

3.4 Ejecutar el script

Ejecuta:

```
python generar_dicom.py
```

Resultado esperado:

```
✓ prueba.dcm generado
✓ Subido a Orthanc: 200
{'ID': 'a3f2c1d4-...', 'Path': '/instances/a3f2...', 'Status': 'Success'}
```

3.5 Verificar en Orthanc

```
curl -u orthanc:orthanc http://localhost:8042/patients
```

Debe devolver un arreglo con al menos un ID, por ejemplo:
`["a3f2c1d4-b5e6f7g8-..."]`

Ese ID es el OrthancPatientId. VigiNurse lo guardará automáticamente en la BD al vincular el paciente. También puedes confirmarlo visualmente abriendo http://localhost:8042 → sección 'Patients'.

Paso 4 — Flujo completo de pruebas con Swagger

Con Orthanc corriendo y el DICOM subido, abre Swagger en http://localhost:5000/swagger y sigue estos pasos en orden:

4.1 Login y autenticación

```
POST /api/auth/login
{ "correo": "tu_usuario@hospital.com", "contrasena": "tu_contraseña" }
```

Copia el token JWT de la respuesta. En Swagger, haz clic en 'Authorize' (candado) y pega:
`Bearer <token>`

4.2 Registrar la configuración PACS

```
POST /api/configuracion-pacs
{
  "hospitalId": 1,
  "nombrePACS": "PACS Principal",
  "orthancUrl": "http://localhost:8042",
  "activo": true
}
```

4.3 Vincular paciente con Orthanc

VigiNurse busca en Orthanc por la cédula del paciente y guarda el OrthancPatientId automáticamente:

POST /api/pacientes/{id}/pacs/buscar

4.4 Ver estudios del paciente (tiempo real)

GET /api/pacientes/{id}/estudios

Copia uno de los OrthancStudyId de la respuesta para el siguiente paso.

4.5 Registrar estudio en la BD

POST /api/ordenes/{id}/estudio

```
{
  "orthancStudyId": "el-id-del-paso-anterior",
  "tipoEstudio": "Radiografía",
  "descripcion": "Rx tórax PA",
  "fechaEstudio": "2026-06-08T10:00:00Z"
}
```

4.6 Ver detalle del estudio

GET /api/estudios/{id}

Combina datos de tu BD con información en tiempo real de Orthanc.

Para limpiar los datos de Orthanc: detén el contenedor (docker stop orthanc-test), borra el contenido de la carpeta db (no la carpeta en sí), e inícialo de nuevo (docker start orthanc-test). Los datos de tu BD no se ven afectados.

Configuración del Visor OHIF para Imágenes Médicas

OHIF (Open Health Imaging Foundation) es el visor de imágenes DICOM que usamos para mostrar radiografías, tomografías y otros estudios directamente desde VigiNurse. Corre como un contenedor aparte y se embebe dentro de la aplicación mediante un iframe.

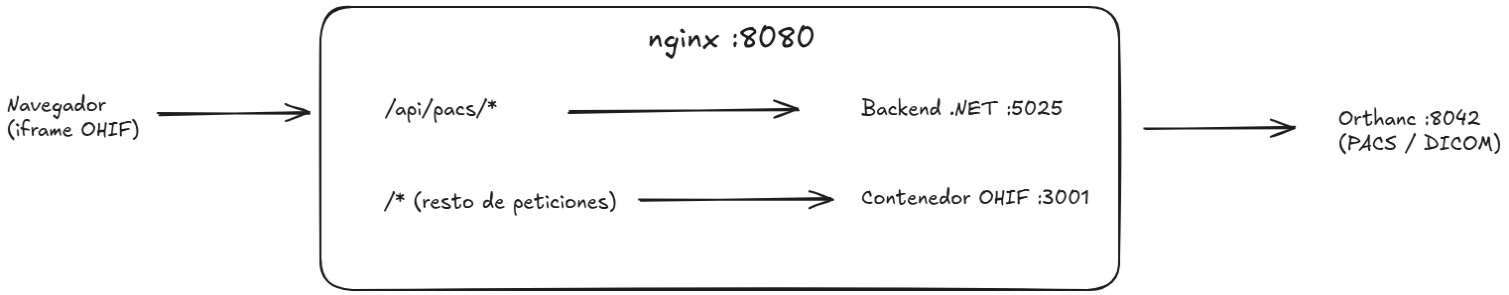
Paso 1 — Configurar nginx como proxy inverso

OHIF y el backend de VigiNurse son dos servidores distintos en dos puertos distintos:

- OHIF (contenedor Docker) → puerto 3001
- Backend .NET (proxy hacia Orthanc) → puerto 5025

Cuando el navegador carga el visor OHIF, el propio JavaScript de OHIF hace llamadas (QIDO/WADO) para pedir estudios, series e imágenes. Si esas llamadas van a un puerto/origen distinto del que sirvió la página, el navegador las bloquea por CORS, y además se pierde el header Referer que necesitamos para identificar el hospital (lo vemos en el Paso 4).

nginx resuelve esto uniendo ambos servidores bajo un solo origen (http://localhost:8080):



Desde el punto de vista del navegador, todo es localhost:8080:

- http://localhost:8080/viewer?... → sirve la interfaz de OHIF
- http://localhost:8080/api/pacs/dicom-web/studies → el mismo OHIF pidiendo imágenes, pero nginx lo reenvía al backend

Sin nginx, tendríamos que configurar CORS en el backend para aceptar localhost:3001, y aun así perderíamos la información del hospital que viaja en el Referer (porque cambiaría el origen entre OHIF y el backend).

1.1 Crear estructura de carpetas

mkdir %USERPROFILE%\nginx-ohif

1.2 Crear el archivo default.conf

Ejecuta este bloque en PowerShell:

```
@"
server {
    listen 8080;

    location /api/pacs {
        proxy_pass         http://host.docker.internal:5025/api/pacs;
        proxy_http_version 1.1;
        proxy_set_header   Host              `$http_host;
        proxy_set_header   X-Real-IP         `$remote_addr;
        proxy_set_header   X-Forwarded-For   `$proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto `$scheme;
    }

    location / {
        proxy_pass         http://host.docker.internal:3001;
        proxy_http_version 1.1;
        proxy_set_header   Host              `$http_host;
        proxy_set_header   Upgrade           `$http_upgrade;
        proxy_set_header   Connection        "upgrade";
    }
}
"@ | Out-File -FilePath "$env:USERPROFILE\nginx-ohif\default.conf" -Encoding UTF8
```

Importante: en Docker Desktop para Windows no se usa --network=host. Por eso localhost dentro del contenedor nginx no apunta a tu PC, hay que usar host.docker.internal, que es la forma en que un contenedor accede a servicios corriendo en el host (el backend en el puerto 5025 y OHIF en el 3001).

1.1 Levantar nginx

```
docker run -d --name nginx-ohif `
  -p 8080:8080 `
  -v "$env:USERPROFILE\nginx-ohif\default.conf:/etc/nginx/conf.d/default.conf:ro" `
  nginx:alpine
```

1.1 Verificar

```
docker ps
```

Debe aparecer nginx-ohif con estado Up, escuchando en el puerto 8080.

Paso 2 — Levantar el visor OHIF

2.1 Levantar el contenedor

```
docker run -d --name ohif `
  -p 3001:80 `
  -e "APP_CONFIG=window.config = { extensions: [], modes: [], showStudyList: false,
studyListFunctionsEnabled: false, defaultDataSourceName: 'dicomweb', dataSources: [{
namespace: '@ohif/extension-default.dataSourcesModule.dicomweb', sourceName:
'dicomweb', configuration: { friendlyName: 'VigiNurse PACS', name: 'dicomweb', qidoRoot:
'http://localhost:8080/api/pacs/dicom-web', wadoRoot:
'http://localhost:8080/api/pacs/dicom-web', wadoUriRoot:
'http://localhost:8080/api/pacs/wado', qidoSupportsIncludeField: false, supportsReject: false,
imageRendering: 'wadors', thumbnailRendering: 'wadors', enableStudyLazyLoad: true,
omitQuotationForMultipartRequest: true } }] };" `
  ohif/app:latest
```

La primera vez Docker descarga la imagen de OHIF (~500 MB), puede tardar varios minutos.

2.2 Verificar

Abre http://localhost:8080 en el navegador. Debería cargar la interfaz de OHIF (vacía, sin estudios, eso es normal porque showStudyList: false).

Paso 3 — Configurar el Frontend

2.2 Variable de entorno

En la raíz del proyecto VigiNurse-FrontEnd, crea (o edita) el archivo .env.local:

```
NEXT_PUBLIC_OHIF_URL=http://localhost:8080
```

Si el servidor de Next.js ya estaba corriendo, hay que reiniciarlo para que tome la nueva variable.

MQTT: Conectar Gateway con Tag

Requisitos previos

- Gateway MKGW3 con cable micro USB o switch POE
- Tag L02 (sensor Bluetooth)
- Teléfono con la app MKScannerPro instalada (App Store y Google Play)
- Gateway y computadora en la misma red WiFi 2.4GHz

Paso 1 — Configurar el Gateway MKGW3

1.1 Encender el gateway

Conecta el cable micro USB a un cargador o puerto USB. También puedes usar un switch POE conectado al puerto POE IN.

1.2 Verificar los LEDs

LED	Color	Significado
Power	Verde fijo	Encendido correctamente
Network	Azul parpadeando	Listo para configurar
Server	Apagado	Sin conexión al servidor aún

Si LED Network no está azul parpadeando: Resetea el gateway: inserta un objeto fino en el agujero de reset y mantén presionado 10 segundos.

1.3 Configurar la app MKScannerPro

Antes de conectarte al gateway, configura la app para que apunte a tu instancia de Mosquitto. Toca el ícono de configuración y completa los campos:

Campo	Valor
Host	192.168.100.7 ← la IP de tu computadora (ipconfigl)
Port	1883
Client Id	app-prueba
Subscribe	(dejar vacío)
Publish	(dejar vacío)
QoS	0
Keep Alive	60

Guarda y verifica que la app diga 'success'. Si dice 'failed', revisa en orden:

- Mosquitto está corriendo (`sc query mosquitto`)
- El archivo mosquitto.conf tiene `listener 1883` y `allow_anonymous true`
- Ejecutaste `net stop mosquitto` + `net start mosquitto` después de editar
- netstat muestra 0.0.0.0:1883 (no 127.0.0.1:1883)

1.4 Conectar al gateway desde la app

- Toca POE Gateway
- Toca Add Devices
- Selecciona el dispositivo MKGW3-XXXX
- Contraseña: Moko4321

1.5 Configurar Network Settings

Campo	Valor
Tipo	WiFi
SSID	nombre de tu red 2.4GHz
Password	tu contraseña WiFi
DHCP	activado

Solo redes 2.4GHz: El gateway MKGW3 no soporta redes 5GHz. Si tu router emite ambas bandas, asegúrate de seleccionar explícitamente la red 2.4GHz al configurar el SSID.

1.6 Configurar MQTT Settings

Campo	Valor
Host	192.168.100.7 ← la IP de tu computadora
Port	1883
Client Id	gateway-prueba
Subscribe	/MKGW3/007007e0ca3c/receive (no borrar, viene por defecto)
Publish	/MKGW3/007007e0ca3c/send (no borrar, viene por defecto)
QoS	0

Keep Alive	60

1.7 Conectar y verificar LEDs

Toca el botón Connect. Los LEDs deben cambiar de la siguiente manera:

LED	Color esperado	Significado
Power	Verde fijo	OK
Network	Verde fijo	Conectado a WiFi
Server	Verde fijo	Conectado a Mosquitto

LED Server en amarillo: Significa que el gateway llegó a la red WiFi pero no pudo conectarse a Mosquitto. Verifica que la IP en MQTT Settings coincide exactamente con la de tu computadora (hostname -I), y que Mosquitto acepta conexiones externas (0.0.0.0:1883).

Paso 2 — Verificar que los datos del Tag L02 llegan

En tu terminal, suscríbete a todos los topics para ver los mensajes en tiempo real:

```
cd "C:\Program Files\mosquitto"
mosquitto_sub.exe -h localhost -t "#" -v
mosquitto_sub.exe -h localhost -t "#" -v | findstr "d3cbfdbd292b"
```

Deberías ver mensajes llegando cada pocos segundos con este formato:

```
/MKGW3/007007e0ca3c/send
{"msg_id":123,"device_info":{"mac":"007007e0ca3c"},"data":[...]}
```

Para filtrar solo los mensajes de tu Tag L02 (reemplaza la MAC por la de tu tag):

```
mosquitto_sub -h localhost -t "#" -v | grep "d3cbfdbd292b"
```

Paso 3 — Conectar el API .NET de VigiNurse

3.1 Librería MQTTnet

El paquete MQTTnet ya está incluido en las dependencias del proyecto. Si necesitas instalarlo manualmente:

```
dotnet add package MQTTnet --version 4.3.6.1152
```

3.2 Configuración en appsettings.json

El bloque Mqtt ya debe estar presente en el archivo. Verifica que la IP y el puerto sean correctos:

```
"Mqtt": {
  "Broker": "192.168.100.7",
  "Port": "1883"
}
```

Entorno local vs producción: Para desarrollo local usa la IP de tu computadora. En producción, usa la IP o hostname del servidor donde corra Mosquitto, y considera habilitar autenticación en lugar de allow_anonymous true.

Paso 4 — Procedimiento al cambiar de red

Cada vez que muevas el equipo o el gateway a una red diferente, sigue estos pasos en orden:

#	Paso
1	Resetea el gateway: inserta un objeto fino en el agujero de reset y mantén presionado 10 segundos.
2	En tu computadora ejecuta `ipconfig` y anota la nueva IP.
3	Verifica que Mosquitto sigue corriendo: `sc query mosquitto`
4	Verifica conexiones externas: `netstat -an | find "1883"`
5	Si muestra 127.0.0.1:1883: edita mosquitto.conf y ejecuta `net stop mosquitto + net start mosquitto`
6	Abre MKScannerPro y actualiza el Host en la configuración de la app con la nueva IP.
7	Conecta al gateway, ve a MQTT Settings y actualiza el Host con la nueva IP.
8	Toca Connect y verifica que los tres LEDs queden en verde.

Resumen de puertos

Servicio	Puerto
VigiNurse API	5025
PostgreSQL	5432
Mosquitto (MQTT)	1883
Orthanc (REST API)	8042
Orthanc (DICOM)	4242
Frontend Web (Next.js)	3000
Frontend Mobile (Expo / Metro bundler)	8081
nginx (proxy OHIF)	8080
OHIF Viewer	3001